

Project Proposal & Methodology

Hybrid RAG & Fine-Tuning for Customer Support

1. Project Scope

1.1.1 Target Use Case

Domain: Automated Tier-1 customer support for the **post-purchase / order-operations** category of an e-commerce business. This scope was chosen because it is the single largest, most repetitive slice of inbound support volume, and it maps directly onto the corporate SOP corpus provided (refund, shipping, returns, account/password recovery, billing, subscriptions, escalation, order tracking, payment methods, data privacy, and working hours).

In scope intents (drawn from the Bitext categories present in the sampled data): order status/tracking, cancellations, refunds, returns, shipping delays, password/account recovery, billing disputes, payment-method changes, subscription cancellation, and complaint escalation.

Out of scope: pre-sales / product-discovery conversations, marketing content, and any action that mutates account or payment state (the system only classifies intent and drafts a policy-grounded response — it never executes a refund, cancellation, or account change itself).

Expected JSON Interface

Input to the fine-tuned intent router (Stage 4):

```
{"query": "the raw, possibly noisy customer message"}
```

Output of the intent router — the structured target the model is fine-tuned to emit:

```
{"intent": "track_order", "category": "SHIPPING"}
```

This structured output is what drives the Hybrid RAG retrieval step (Task 4.3): the intent is mapped to a specific SOP search string instead of the raw, noisy query.

1.1.2 Success Criteria

Quantitative targets, defined *before* development began, evaluated on the held-out test split described in Stage 2:

Metric	Baseline (expected)	Target for Hybrid RAG (V2)
Format Adherence Rate	n/a (free text)	≥ 90% valid JSON
Intent Accuracy (exact match)	n/a	≥ 75% on test split, ≥ 60% on adversarial subset
ROUGE-1 / ROUGE-L	low (ungrounded)	Improve over Baseline and V1 (Naive RAG)
BLEU	low (ungrounded)	Improve over Baseline and V1 (Naive RAG)
Consistency Rate	100% (greedy)	100% (greedy, do_sample=False)
Hallucination Frequency	high (unconstrained)	≥ 40% relative reduction vs Baseline

These targets are re-measured identically for Baseline, Solution V1, and Solution V2 in the Comparative Analysis Report, so improvement is attributable and quantified stage-by-stage.

2. Proposed Methodology

1.4.1 Baseline Model Selection

Selected model: Qwen/Qwen2.5-1.5B-Instruct.

- **Hardware constraints:** the assignment targets a free-tier T4 GPU (Colab); this project was additionally executed end-to-end locally on Apple Silicon (M-series, MPS backend, no CUDA). At 1.5B parameters the model loads comfortably in fp16 on either environment without requiring the 8B-class model's >16GB VRAM footprint.
- **Dataset characteristics:** the Bitext-derived instruction set (~1,500 rows after cleaning) is small; a 1.5B model has enough capacity to learn the JSON-extraction task via LoRA without the severe overfitting risk a larger model would carry on so little data.
- **Fine-tuning feasibility:** Qwen2.5-Instruct ships with a native ChatML template (`apply_chat_template`) and strong out-of-the-box structured-output behaviour, which materially improves LoRA sample-efficiency for the JSON-router task.
- **Rejected alternatives:** TinyLlama-1.1B-Chat is lighter but empirically weaker at emitting strict JSON; Llama-3-8B-Instruct gives the best raw quality but its memory footprint (>16GB even 4-bit) is infeasible on the constrained hardware used here.
- **Hardware adaptation note:** because execution is on Apple Silicon rather than a CUDA GPU, bitsandbytes 4-bit quantisation (CUDA-only) is unavailable. The model is instead loaded in fp16 directly on the MPS device — functionally equivalent for this project's purpose of demonstrating the RAG + PEFT pipeline, and explicitly documented as a deviation from the Colab/T4 reference environment.

1.4.2 Retrieval Strategy

- **Embedding model:** sentence-transformers/all-MiniLM-L6-v2 (384-dim, CPU-friendly, ~80MB) — used identically across NB4, NB5, and NB7 to keep retrieval results comparable across evaluation stages.
- **Chunking:** each corporate SOP Markdown file is treated as a single retrieval unit (documents are short, 150-250 words each, single-topic) — no further sub-chunking is required at this corpus size.
- **Vector storage:** ChromaDB, persisted to `./chroma_db`, one collection for the full SOP corpus.
- **Similarity search:** naive RAG (V1) embeds the raw customer query and retrieves the top-1 nearest document ($k=1$). Hybrid RAG (V2) instead searches using the structured intent string extracted by the fine-tuned router, which is far less noisy than the raw query.
- **Retrieval evaluation:** validated directly (does the retrieved document match the query's true policy area?) and indirectly, via the downstream ROUGE/BLEU and hallucination-frequency scores of the generated answer.

1.4.3 Fine-Tuning Strategy

- **Technique:** LoRA (not QLoRA) via peft — QLoRA's 4-bit quantisation path depends on bitsandbytes, which requires CUDA and is unavailable on the Apple Silicon execution environment used here.
- **Target modules:** `q_proj`, `v_proj` (attention projections) — updates ~0.5-1% of total parameters.
- **Adapter config:** rank $r=16$, `lora_alpha=32` ($2\times r$), `lora_dropout=0.05`, `task_type=CAUSAL_LM`.
- **ChatML structure:** system + user (raw query) + assistant, where the assistant turn is forced to a strict JSON string `{"intent": ..., "category": ...}` generated via `json.dumps`, with -100 label-masking applied to padding tokens so loss is computed only on real content.

- **Training loop:** custom PyTorch loop with AdamW ($lr=2e-4$), batch size 4, validation every VAL_EVERY steps, early stopping on validation loss (PATIENCE=3).
- **Checkpointing / logging:** best-val-loss checkpoint saved to ./intent_lora_best/, final checkpoint to ./intent_lora/, plus a training_log.csv and loss-curve plot as reproducibility evidence.

1.4.4 Evaluation Framework

- **Format Adherence Rate:** percentage of router outputs that parse successfully with `json.loads()`.
- **Intent Accuracy:** exact-match rate of the extracted intent field against ground truth, plus a fuzzy-match variant (e.g. `order_tracking` vs `track_order`) to capture near-misses.
- **ROUGE-1 / ROUGE-L / BLEU:** computed against SOP-grounded reference answers (not the generic Bitext response column), so policy-specific language is rewarded.
- **Consistency Rate:** repeated greedy-decoded generations (`do_sample=False`, `temperature=None`) for the same input must be identical.
- **Hallucination Frequency:** percentage of generations containing claims (dates, numbers, policy terms) not supported by the retrieved SOP context, checked for Baseline, V1, and V2 identically.
- All five metrics are re-computed under an identical configuration (same MODEL_ID, same embedding model, same test split) across all three system versions so the deltas are attributable.